

test_scenario_aop.vr

\$VCS_HOME/doc/examples/nativetestbench/openvera/rvm/scenarios/test_scenario_aop.vr

```
//
// Demonstrate how to use scenarios kinds
// i.e. each scenario defined as a different
// variations of a single descriptor
//
#include "scenarios.vrh"
//
// Telegraph good & bad cells
// good - BAD - good - BAD - good
// VPIa  VPIb  VPIa  VPIb  VPIa
//
extends my_atm_cell_scenarios(atm_cell_atomic_scenario) {
    integer BAD_CELLS;
    integer PROCEDURAL;

    constraint bad_cells_scenario {
        if (kind == BAD_CELLS) {
            length == 5;
            repeated == 0;
            foreach (this.items, i) {
                if (i % 2) this.items[i].hec != 8'h00;
                else      this.items[i].hec == 8'h00;
                if (i > 1) this.items[i].vpi == this.items[i-2].vpi;
                if (i == 1) this.items[i].vpi != this.items[i-1].vpi;
            }
        }
    }
    constraint procedural_scenario {
        if (kind == PROCEDURAL) {
            length == 0;
            repeated == 0;
        }
    }
    constraint hec_valid {
        if (kind != this.BAD_CELLS) {
            foreach (this.items, i) {
```

```

        this.items[i].hec == 8'h00;
    }
}
}
after task new() {
    this.BAD_CELLS = define_scenario("Telegraph bad cells", 5);
    this.PROCEDURAL = define_scenario("Procedural cells", 0);
}
after task pre_randomize() {
    foreach (this.items, i) {
        void = this.items[i].constraint_mode(OFF, "hec_valid");
    }
}
around virtual function integer apply_t(atm_cell_channel channel) {
    if (kind == this.PROCEDURAL) {
        apply_t = this.procedural_scenario_t(channel);
        return;
    }
    proceed;
}

function integer procedural_scenario_t(atm_cell_channel channel) {
    atm_cell cell = new;

    void = cell.randomize() with {
        gfc == 4'h0;
        vpi == 8'h00;
        vci == 16'h0000;
        pt == 3'h0;
        clp == 1'b0;
    };

    repeat (3) {
        atm_cell c;
        cast_assign(c, cell.copy());
        channel.put_t(c);
    }

    void = cell.randomize() with {
        gfc == 4'hF;
        vpi == 8'hFF;
        vci == 16'hFFFF;
    };
}

```

```
        pt == 3'h7;
        clp == 1'b1;
    };

    channel.put_t(cell);

    procedural_scenario_t = 4;
}
}
```